

Technical Documentation: Real-Time UAV Image Stitching Pipeline

This document provides a comprehensive technical specification of the image-only sequential image stitching pipeline designed for resource-constrained embedded platforms, specifically the Raspberry Pi 5. The architecture optimizes computational efficiency through adaptive region cropping and localized blending without relying on external telemetry such as GPS.

1. Pipeline Architecture Overview

The stitching pipeline processes a sequence of aerial images captured by a UAV down-facing camera. It dynamically builds a continuous mosaic map by registering each incoming frame against a running canvas. The core objective is to minimize CPU cycles during feature extraction, which represents the primary computational bottleneck in computer vision pipelines on ARM architectures.

2. Detailed Component Specifications

2.1. Preprocessing and Stabilization

To reduce memory bandwidth and processing latency, raw high-resolution images undergo controlled downsampling and initial geometric stabilization before entering the core registration loop.

- **Downsampling:** Images are spatially downsampled by a factor of 5 using strided array slicing, significantly reducing the pixel count while preserving essential structural layouts.
- **Un-rotation:** Using an attitude matrix derived from pitch, roll, and yaw values provided in the companion data matrix, an un-rotation matrix is computed. The downsampled image is then rectified via perspective warping with padding to compensate for camera tilt and attitude variations.
- **Memory Management:** To prevent RAM exhaustion on the embedded system, raw image matrices are explicitly cleared (``None``), and the Python garbage collector (``gc.collect()``) is systematically invoked every 5 frames.

2.2. Adaptive ROI Estimation (Novelty B1)

The primary optimization mechanism is the Adaptive Region of Interest (ROI) Estimation. Instead of running feature detection on the full frame of the incoming image (I_n), the pipeline uses the relative transformation matrix from the previous frame pair (H_{rel_prev}) to predict where the overlapping region will appear.

Assuming relatively stable forward camera motion, the transformation matrix mapping I_n

I_{n-1} is inverted via matrix inversion ($H_{inv} = H_{rel_prev}^{-1}$) to project the boundaries of the previous frame onto the coordinate space of the incoming frame. The projection maps the 4 corner points of the frame canvas:

```
corners = [[0, 0], [0, height], [width, height], [width, 0]]  
warped_corners = cv2.perspectiveTransform(corners, H_inv)
```

An axis-aligned bounding box is calculated by taking the minimum and maximum extremes of the warped corners (`x_min`, `y_min`, `x_max`, `y_max``). To ensure robustness against wind disturbance and sudden platform jitter, a margin of tolerance (padding) equal to 15% of the image dimensions is added around the bounding box. The coordinates are clamped to ensure they remain within valid image array boundaries $[0, \text{width}]$ and $[0, \text{height}]$.

Fallback Mechanism: If the predicted ROI dimensions are calculated to be invalid or excessively small (less than 100 pixels in width or height), the system automatically aborts the crop and defaults to processing the full frame to avoid catastrophic tracking loss.

2.3. Localized Feature Extraction and Description

Feature extraction utilizes the Scale-Invariant Feature Transform (SIFT) algorithm, configured to detect a maximum of 1,800 keypoints to balance performance and feature density.

- **Masked Extraction:** The incoming frame is sliced according to the predicted ROI boundaries (`image2_roi = image2[y_start:y_end, x_start:x_end]`). SIFT feature detection is executed exclusively on this sub-array. A binary threshold mask is applied to ignore invalid black borders introduced during rectification.
- **Coordinate Space Remapping:** Keypoints detected within the sub-array crop are localized to the ROI's relative coordinate space (starting at $(0,0)$). To maintain consistency during global transformation estimation, the coordinates of these keypoints are remapped back to the full image space by adding the spatial offsets (`x_start`, `y_start``) during keypoint object reconstruction.

2.4. Feature Matching and Filtering

Keypoint descriptors from the stabilized historical image (I_{n-1}) and the remapped incoming image (I_n) are matched using a Brute-Force Matcher (`cv2.BFMatcher`). To discard ambiguous and erroneous correspondences, the pipeline applies Lowe's Ratio Test on the top 2 nearest neighbors ($k=2$). A strict distance ratio threshold of 0.55 is enforced; matches are only accepted if the closest distance is less than 55% of the second-closest neighbor's distance.

2.5. Transformation Estimation

The pipeline employs a dual-stage geometric transformation estimation strategy to achieve maximum stability and computational speed depending on the terrain structure.

Estimation Stage	Algorithm / Function	Operational Intent & Output
Primary Stage	Partial Affine Transform (`cv2.estimateAffinePartial2D`)	Attempts to resolve translation, rotation, and uniform scaling. Highly stable on planar surfaces with low degree-of-freedom changes. Returns a 2×3 matrix.
Secondary Fallback	Homography Estimation (`cv2.findHomography` with RANSAC)	Executed if the affine model fails. Resolves full perspective distortion (8 degrees of freedom) using random sample consensus to filter outlying feature matches. Returns a 3×3 matrix.

If an affine matrix is successfully resolved, it is converted into a homogeneous 3×3 matrix by vertically stacking an identity row `[0, 0, 1]` to enable uniform matrix multiplication in the subsequent chaining phase.

2.6. Accumulated Homography Chaining

To stitch images sequentially without executing computationally intensive bundle adjustment over a global keyframe graph, the relative transformation matrix (H_{rel}) is accumulated into a running global transformation matrix ($H_{\text{global_current}}$) via chained matrix dot-multiplication:

```
H_global_current = np.dot(self.H_global_prev, H_rel_3x3)
H_global_current = H_global_current / H_global_current[2, 2]
```

The matrix is scaled by dividing by its bottom-right element ($H_{\{2,2\}}$) to maintain numerical normalization across successive iterations. The accumulated matrix represents the direct spatial mapping from the current frame to the initial coordinate system of the mosaic anchor canvas.

2.7. Warping and Localized Feather Blending

The canvas boundary dimensions (`xMin`, `yMin`, `xMax`, `yMax`) required to enclose both the existing mosaic and the new transformed image are dynamically computed. Both images are then warped onto the expanded canvas space. To eliminate visible seams and illumination discrepancies without incurring high memory overhead, the pipeline uses a specialized *ROI Feather Blender* (`ROIfeatherBlender._roi_feather_blend`). This blender calculates alpha-blending weight masks exclusively within the intersecting region of interest, optimizing pixel blending arithmetic and preventing memory bottlenecks.

3. Software Design & Class Structure

The software pipeline is encapsulated within the `Combiner` class. The internal execution layout is structured as follows:

- `__init__`: Initializes tracking states, sets `H_global_prev` to the identity matrix, and invokes image preprocessing.
- `__preprocess_images`: Performs downsampling, sensor-driven un-rotation, and garbage collection.
- `__predict_roi`: Implements the geometric projection and boundary slicing for Adaptive ROI.
- `__detect_features` & `__match_features`: Handles masked SIFT extraction, remapping, and ratio-tested matching.
- `__estimate_transform` & `__compute_canvas_bounds`: Calculates relative motion models and canvas expansions.
- `combine`: Coordinates the single-step execution flow for a frame pair, updates states, and exports intermediate results.
- `create_mosaic`: Executes the sequential loop across the image list and outputs the final stitched orthomosaic.